

SHREE GURU GOBIND SINGH TRICENTENARY UNIVERSITY

Gurugram, Haryana

Department of Computer Science & Engineering

TERM PAPER · THEMATIC ASSESSMENT, PHASE 2

Database Management Systems Laboratory

Persistent Memory Architecture in Agents Using DBMS

Comparison, Findings and Recommendation

Submitted by **Harshit Khemani** 241302081

Kush Ahuja

Madhav Bassi

Kushagra Agrawal

Programme B.Tech, Computer Science & Engineering (AI/ML)

Section C

Department Computer Science & Engineering

Institution SGT University

Date of submission 10 September 2026

Abstract

Phase 1 of this term paper argued that the memory of an AI agent is a database, and it described how four deployed systems keep facts across a session boundary. Phase 2 tests that argument against measurement.

The subject of this phase is **Lore**, the open-source memory system that Notion released for MCP-compatible assistants [4, 5]. Lore is a good subject because it is honest about its own trade. It stores agent memory as rows in five Notion databases, so the memory stays readable by people and needs no server. In exchange it gives up every guarantee a database declares: no unique index, no foreign key, no check constraint, no atomic write across pages, no join, no aggregate and no conditional write.

This report compares that design against two relational engines that run the same schema, reports what the comparison found, and gives one recommendation. The evidence comes from our companion study, which reconstructed Lore’s 80 properties from its source, generated a vault of 11 885 memories and 4 277 facts with definitional ground truth, and ran a workload of 361 questions in ten classes through three stores [7].

The result is not that the document store gives wrong answers. It gives right ones. It answers 100.0% of the workload exactly, and so does SQLite; PostgreSQL answers 99.2%, and its three divergences are a stemming rule inside its text index rather than a fault. What separates the three is *cost* and *refusal*. The same 361 questions cost the Notion Data API 522 686 requests against 361 SQL statements. Eight agents writing one key through a read-then-write protocol lose 87.1% of their updates, and the same workload through a unique index loses none. PostgreSQL declines even to add a temporal exclusion constraint to the vault, because 70 subject–predicate pairs already contradict themselves.

The recommendation follows from those numbers, and it is not “use PostgreSQL”. It is to keep the legible store as the system of record and to add a rebuildable relational projection beside it, which carries the indexes, the constraints and the joins the legible store cannot declare.

KEYWORDS

agent memory, request amplification, lost update, referential integrity, bitemporal data, exclusion constraint, normalisation, entity resolution, PostgreSQL, SQLite.

WHAT THIS REPORT DOES NOT REPEAT

This is a standalone Phase 2 report. No section, table, figure or argument from the Phase 1 report appears here. Phase 1 covered the four storage layers, the PostgreSQL schema, the record and recall paths, and four case studies. This phase covers only the three criteria set for it: **comparison**, **findings** and **recommendation**. It takes a new subject system, a new evidence base, and a bibliography that shares no research paper with Phase 1. Where a Phase 1 conclusion is needed, it is named in one sentence, never reproduced.

New Research Added in This Phase

The brief for this phase asks for three more research papers. The three below are new to this report, none of them appears in Phase 1, and each does real work in the sections that follow rather than sitting in the list. Table 1 names what each one contributes and where it is used.

Table 1. The three research papers added in Phase 2, and the work each one does.

Paper	What it contributes here	Used in
Zep , a temporal knowledge graph for agent memory [1, 8]	Zep attaches a validity interval to every subject–predicate–object edge and invalidates an old edge instead of deleting it. It is the sharpest prior art for Lore’s Facts relation, so it fixes the axis on which the comparison of data models is fair.	§1, §2.4
Mem0 , scalable long-term memory with explicit operations [2]	Mem0 gives memory a small data manipulation language: the extractor decides ADD, UPDATE, DELETE or NOOP for every candidate. That names the axis Phase 1 left open, which is who owns the update and where it is applied.	§1, §2.6
LongMemEval , a benchmark for long-term interactive memory [3]	It isolates five abilities of a long-term memory, two of which — temporal reasoning and knowledge updates — are exactly the two a validity interval exists to serve, and it measures about a 30% accuracy drop for commercial assistants across sustained interactions. That ties the validity interval to abilities a benchmark treats as core.	§1, §2.4

Six further sources are also new to this report. Five support the argument at specific points: Graphiti, the engine behind Zep [8], the SQL:2011 temporal standard [9], Gray’s account of the lost update [10], Swoosh on entity resolution [11], and a recent survey that maps the operations of an agent memory onto the operations of a database [13]. The sixth, A-MEM [12], states the opposing position on whether a memory should have a schema at all.

Contents

Abstract	ii
New Research Added in This Phase	iii
1 Comparison	1
1.1 The subject, and where the numbers come from	1
1.2 Three agent memories, compared as data models	1
1.3 One schema, three stores	2
2 Findings	3
2.1 A missing join costs a multiplier, not a constant	3
2.2 A read-then-write upsert loses most of its writes	4
2.3 A constraint the store cannot declare is a state it cannot refuse . . .	4
2.4 Two time axes disagree often enough to be a correctness decision . .	5
2.5 The normalisation violations that ship are substrate costs	5
2.6 The update anomaly lands in the prose, not in the table	6
2.7 Three indexes over one text give two different answers	6
3 Recommendation	7
References	8

List of Figures

1 Request amplification by question class	3
---	---

List of Tables

1 The three new research papers	iii
2 Three memory designs	1
3 The three arms	2
4 The seven invariants	4

1 Comparison

Phase 1 asked how an agent memory carries a fact across a session boundary. That separates designs but does not price them. This phase asks a narrower question with a harder answer. *One schema, three stores: what does each cost, and what will each refuse?*

1.1 The subject, and where the numbers come from

Lore is Notion’s open-source memory system for MCP-compatible assistants [4, 5]. A vault is one Notion page holding five databases — Projects, Topics, Memories, Entities and Facts — all reached through the Notion Data API. That API is the whole design: at most 100 rows per request, 3 requests per second, and no join, no aggregate, no unique index, no check constraint and no conditional write [6].

Every number here comes from our companion study [7], which reconstructed Lore’s 80 properties from `src/notion/schema.ts` at commit 95c3558 and re-implemented the schema on SQLite and on PostgreSQL. Its corpus is generated world-first, so every correct answer is definitional: 11 885 memories, 4 277 facts of which 1 196 are closed, 1 096 entities carrying 1 153 aliases. A workload of 361 questions in ten classes then runs through three arms: an emulator restricted to the documented Data API, and the two engines.

1.2 Three agent memories, compared as data models

A comparison needs a fair axis, and the three new papers supply it. Zep attaches a validity interval to every edge of a temporal knowledge graph, so an old claim is invalidated and not deleted [1, 8]. Mem0 gives the writer explicit operations, so a candidate becomes an ADD, an UPDATE, a DELETE or a NOOP [2]. Lore sits between them, and Table 2 sets the three side by side.

Table 2. Three agent memory designs as data models. The last row is the one the rest of this report measures.

	Lore [4]	Zep and Graphiti [1, 8]	Mem0 [2]
Temporal model	Bitemporal. Valid From, Valid Until, Observed At, Invalidated At	Bitemporal. An edge records when it was true and when the graph learned it	Timestamps on the item. No validity interval
How an update happens	The client reads, decides, then writes. Two requests, and a window between them	The engine closes the old edge and adds the new one	An explicit operation chosen at write time: ADD, UPDATE, DELETE or NOOP
The store can refuse a bad write	No. Uniqueness is a convention in the client	Yes, for what the engine declares	Only as far as the writer decides

One row is missing on purpose: only Lore is readable and correctable by a person without a client, the trade §3 defends. A-MEM argues the opposite case to all three, that a fixed schema limits cross-task adaptability [12]; this phase does not settle that, but none of the failures in §2 follows from having a schema. Each follows from a store that cannot enforce one. LongMemEval justifies the temporal row: two of the five abilities it isolates, temporal reasoning and knowledge updates, are exactly the two a validity interval exists to serve [3].

1.3 One schema, three stores

Table 3 is the centre of this comparison: the same schema, the same 361 questions, three stores.

Table 3. The three arms on one workload of 361 questions. Requests and client-side rows are means for one question. The rate-limit floor applies Notion’s documented 3 requests per second to the whole workload; it is a lower bound on wall-clock, not a measurement of it, and no rate limit applies to a local engine. The PostgreSQL arm is PGLite, the same source tree and planner compiled to WebAssembly, so no throughput claim is made for it.

Measure	Lore on Notion	SQLite	PostgreSQL
Exact-answer rate	100.0%	100.0%	99.2%
Requests, one question	1 447.9	1.0	1.0
Requests, whole workload	522 686	361	361
Rate-limit floor, whole workload	48.4 h	—	—
Share of the workload it can state	33.2%	100%	100%
Invariants with no declarative form	4 of 7	1 of 7	0 of 7

The last two rows need a note. The first counts what the store can *express*: 44.6% of the questions need a join, 11.4% an aggregate and 11.1% a transitive closure, so 66.8% of the workload needs something the Data API cannot state as a filter. The second counts what it can *refuse*: four of the seven invariants have no declarative form on Notion and three of those are guarded by nothing at all, while SQLite declares six, failing only on the temporal one for want of an exclusion constraint, and PostgreSQL declares all seven. Figure 1 in §2.1 draws the uneven distribution behind the 1 447.9 mean.

So the comparison does not separate the three on correctness. It separates them on cost, as a multiplier rather than a constant, and on refusal, which is categorical: a database’s central service is not answering a question but declining to enter a state that makes a future answer wrong.

2 Findings

Most of the findings below carry the name of an older database phenomenon, because that is what they turn out to be.

F1 One result frames the rest. **Correctness is not where these designs differ**, so nothing below is about a wrong answer. All three stores return the exactly-correct set, and PostgreSQL’s 99.2% is the stemming rule of §2.7, not a fault. Everything below is about what the right answer costs and about what the store will not stop.

2.1 A missing join costs a multiplier, not a constant

F2 Provenance is the cheapest join an agent memory can be asked for: a claim, the memory it came from, and that memory’s author. In SQL it is one statement. The Data API cannot filter one database by a property of the page a relation points at, so the client pages over Facts in 12 requests, reads 1 126 memory pages one at a time and filters itself — 1 137.75 round trips and 1 126.2 client-side rows against one and none, a floor of 6.3 minutes for one question.

Searching what the memories say is worse. Property filters cannot see page blocks and the search endpoint matches titles [6], so a complete search is one list request per hundred rows plus one per row: 11 892 requests, 66.1 minutes here, 11.0 hours ten times larger, 110.1 hours a hundred times larger.

That figure carries the study’s own biggest caveat: it measures what the *documented* API permits, and Lore delegates ranking to a Notion endpoint the study could not measure, so Lore is cheaper here [7]. The structural point survives — Lore owns no index, so its retrieval depends on an endpoint it does not control.

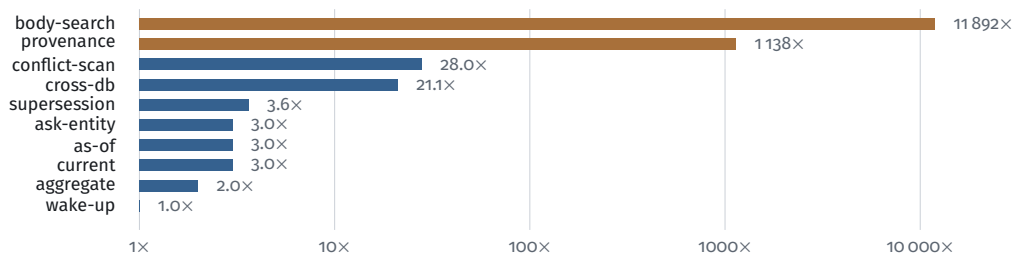


Figure 1. Request amplification by question class: the mean number of Data API requests that one question costs, against the single SQL statement which answers it. The axis is logarithmic and evenly spaced by decade, so wake-up at 1x has no bar to draw. The two clay bars are the two classes that pass a thousand requests for one question, and both are searches the store cannot push down.

F2b The prompt is bounded and the work behind it is not. The wake-up hook caps its payload at 20 memories and 30 facts, and the cap holds: over an eight-fold growth of the vault it still costs 24 requests and injects between 775 and 852 tokens. What grows is the class of operation touching every row — the conflict

scan, the debt sweep, the aggregate, the complete search — which a rate limit turns into hours. Lore’s own caps cluster there.

2.2 A read-then-write upsert loses most of its writes

F3 Lore upserts a topic-keyed memory by reading the vault for a matching row and then creating or patching. The API documents no conditional write — no ETag, no If-Match, no version token [6] — so the window between read and write cannot be closed from the client, and Lore’s own source says so. The study ran the interleave explicitly rather than leaving it to a scheduler: every writer reads before any writer writes, the worst schedule the protocol admits and the one the API gives no way to exclude. With 8 writers over 200 rounds the read-then-write path lost 1 393 of 1 600 updates, 87.1%, and left 7 duplicate rows for a key that should hold one. The same workload through `INSERT . . . ON CONFLICT` lost none.

The loss grows with contention. A shallower sweep at 2, 4, 8 and 16 writers lost 49.0%, 73.5%, 85.8% and 91.9%, against 0.0% at every width through the unique index. This is Gray’s lost update [10] by a new route: the anomaly needs no weak isolation level when the protocol has no conditional write to close on.

2.3 A constraint the store cannot declare is a state it cannot refuse

F4 The study took seven invariants from the schema and wrote the one line of standard DDL that would enforce each (Table 4). Four have no declarative form on the substrate at all, and three of those are guarded by nothing whatsoever.

Table 4. Seven invariants the schema implies, the declaration that would enforce each, and what walks through it today.

	Invariant	Declarative form	In the vault
INV-1	At most one memory for one topic key	<code>UNIQUE (topic_key, project_id)</code>	Client code
INV-2	Every fact points at a memory that exists	<code>FOREIGN KEY ... ON DELETE RESTRICT</code>	Nothing
INV-3	One object per functional predicate at any instant	<code>EXCLUDE USING gist (... WITH &&)</code>	Nothing
INV-4	Confidence is one of three values	<code>CHECK (confidence IN ...)</code>	Editable options
INV-5	An alias names exactly one entity	<code>UNIQUE (alias)</code>	Nothing
INV-6	Valid from is not after valid until	<code>CHECK (... <= ...)</code>	Not verifiable
INV-7	The dedup key is the hash of its own columns	<code>GENERATED ALWAYS AS (...)</code>	Write-time code

Three were attacked. INV-1 is F3 above; in the other two the vault refused nothing at all. Over 2 724 facts on functional predicates, PostgreSQL under a temporal exclusion constraint accepted 2 532 and refused 192, and the vault refused 0. When

112 memories were archived after facts already cited them, ON DELETE RESTRICT refused all 112 and left no orphan; the vault accepted all 112 and left 112 claims whose provenance it can no longer return. The exclusion result has a sharper first half: the constraint could not be added to the vault *as generated* at all, because 70 subject–predicate pairs already assert two objects over overlapping time.

Lore is not unaware of contradictions — it ships a scanner, capped at 500 candidates for each project. The comparison is between preventing a state and detecting it afterwards at quadratic cost under a cap. Detection is useful, and it is not the same thing.

2.4 Two time axes disagree often enough to be a correctness decision

F5 A bitemporal schema is only worth its columns if the two axes ever disagree. Here they disagree constantly. A retraction lags the world by a median of 23 days and a 95th percentile of 43, and of the 1 196 facts whose validity has closed, 1 172 — 98.0% — were retracted only after the claim had stopped being true. A fact disagrees on day D when “was this true on D ” and “did the vault believe it on D ” give different answers; across all 4 277 facts, 245 disagree on day 120 (5.7%), 123 on day 240 (2.9%), 112 on day 360 (2.6%) and 55 on day 480 (1.3%).

Lore and Zep both hold the two axes [4, 1], and SQL:2011 standardised the separation more than a decade before either [9]. LongMemEval measures what getting it wrong costs: assistants lose about 30% of their accuracy across sustained interactions [3]. A query that does not name its axis is a correctness error waiting for a probe day.

2.5 The normalisation violations that ship are substrate costs

F6 The usual exercise walks a badly shaped relation up to Boyce–Codd form. The interesting exercise here is the other one: which violations *ship*, and why. Five rungs of a six-rung ladder are still in the released product, and none is an oversight. Aliases are a comma-joined list in one text cell, so no predicate can ask which entity owns an alias; a multi-select would force a schema migration for every new alias. Two derived columns stand in for one expression index, because the store has none. And the centre of the knowledge graph carries a Boyce–Codd violation that is structural rather than careless: every Notion database needs exactly one title property, a title cannot be a relation, so a fact holds its subject twice.

That violation is reachable by the most ordinary maintenance the system has. The merge routine repoints every fact relation from loser to winner and never rewrites the title, and the vault was generated to hold what that leaves behind: 105 facts, 2.5% of all, whose title says one name and whose relation points at another. Entity

resolution has an older theory — Swoosh defines the transitive closure a resolution must reach [11] — and Lore’s merge is careful but single-hop, so 39 subjects have their facts split across two rows.

One measurement went the other way. The study predicted that resolving an alias through the joined cell would be expensive, because a substring test over-matches, and built 57 aliases as deliberate prefixes of others. Across 1 057 alias terms the filter returned 1 159 rows of which 1 153 were exact: six discarded, 0.5% of the traffic. The magnitude did not support the argument, so it is reported as measured.

2.6 The update anomaly lands in the prose, not in the table

F7 This is the finding a schema alone does not fix. A fact is restated across several memories in prose, and when the fact changes the Fact row is updated and the prose is not. Across the 1 196 closed facts, 3 867 memories still assert the closed value: a mean of 3.23 stale restatements each, and for 33 facts the stale copies alone would fill a ten-row recall. An agent that retrieves memories rather than facts retrieves the old answer, in a confident sentence, nothing marking it retired.

This is the classical update anomaly. Mem0 answers it by making UPDATE and DELETE explicit operations a writer must choose [2]. Lore supersedes the fact row, which is correct and incomplete, because the copies live where supersession does not reach.

2.7 Three indexes over one text give two different answers

F8 The workload turned up a result it was not designed to find. Seven test phrases returned identical counts on all three indexes — *call graph* matched 677 memories on each, *error rate* 200, *deploy manifest* 152. The eighth did not. *Timed out* appears nowhere as a literal string, because the memories say *times out*, so a literal search and SQLite’s FTS5 both return 0 and PostgreSQL returns 669: the English configuration stems *timed* to *time* and drops *out* as a stop word, so the phrase collapses to the lexeme ‘time’ and matches every memory that mentions timing at all.

None of the three is wrong — a stemmer is a considered position on what “discusses this” means. What matters is that the position is a choice and no answer states which choice was made, so two teams on two engines reason from two different sets of facts.

3 Recommendation

The recommendation is not “use PostgreSQL”. Moving off the document substrate forfeits three things that are not conveniences: a surface non-engineers can audit, an authorisation model already provisioned, and no operational estate at all.

Keep the legible store as the system of record, and put a relational projection in front of it. The projection holds the indexes, the constraints and the joins the substrate cannot declare, and holds no authority: it derives from the vault, so if it is wrong, drop it and derive it again. Because a constraint cannot refuse a write it never sees, the write paths differ. **The agent writes through the projection**, where a bad write is refused before it reaches the vault, which is what makes rules 1 and 3 true rather than aspirational. **People still write straight to the vault**, so a human edit that breaks an invariant fails the next rebuild instead — detection rather than prevention, on a schedule nobody forgets. **Every read goes to the projection**, where a join, an aggregate and a transitive closure are each one statement.

Six rules follow. Each names the finding forcing it; F1 forces none, being the framing result rather than a defect.

1. **Declare the key where the data is.** A unique index and one ON CONFLICT statement close the window a read-then-write upsert cannot; the same index on an alias stops one real thing being introduced twice. (*F3, F4, F6*)
2. **Put the join on the engine’s side of the wire.** Move the classes needing a join, an aggregate or a transitive closure — two thirds of this workload. (*F2*)
3. **Give the fact table a temporal primary key.** An exclusion constraint over a range type refuses the contradiction, leaving the scan nothing to find [9]. (*F4, F5*)
4. **Make every query name its axis.** Was the claim true on day *D*, or did the store believe it on *D*? (*F5*)
5. **Retire the restatements when you retire the fact.** Give the extractor an explicit UPDATE and DELETE, as Mem0 does [2]. (*F7*)
6. **Record what answered, and log what a cap dropped.** A silent truncation reads as complete coverage. (*F2b, F8*)

When, and with what caveat. Not on day one, but where maintenance passes stop finishing inside a session or more than one agent writes one key. The measurements bound the shape of the cost rather than any team’s wall-clock, coming from an emulator over a generated corpus [7]. What they settle is firmer: a memory that arrives at triples with validity intervals, an entity registry, provenance edges and a confidence score has arrived at a database schema, and the only question left is which guarantees it does without, and what it does instead.

References

Entries [1]–[3] are the three research papers added in this phase. Entries [4]–[7] are the subject system and the evidence base. Entries [8]–[13] support the argument and are also new to this report. No entry is carried over from Phase 1.

- [1] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan and D. Chalef. “Zep: A Temporal Knowledge Graph Architecture for Agent Memory.” arXiv:2501.13956, 2025.
<https://arxiv.org/abs/2501.13956>
- [2] P. Chhikara, D. Khant, S. Aryan, T. Singh and D. Yadav. “Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory.” *ECAI 2025*, Frontiers in Artificial Intelligence and Applications, vol. 413, pp. 2993–3000, 2025; arXiv:2504.19413. <https://arxiv.org/abs/2504.19413>
- [3] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang and D. Yu. “LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory.” *ICLR 2025*; arXiv:2410.10813. <https://arxiv.org/abs/2410.10813>
- [4] Notion Labs, Inc. (makenotion). “Lore — persistent, shared AI memory backed by Notion for MCP-compatible assistants.” GitHub, MIT licence, read at commit 95c3558. <https://github.com/makenotion/lore>
- [5] H. Salman. “Building Shared Memory for AI Agents in Notion.” Notion Blog, 18 August 2026.
notion.com/blog/building-shared-memory-for-ai-agents-in-notion
- [6] Notion Labs, Inc. “Request Limits”, “Pagination” and “Search by Title.” Notion API Reference. <https://developers.notion.com/reference/request-limits>
- [7] H. Khemani, K. Ahuja, M. Bassi and K. Agrawal. “Persistent Memory Architecture for Agents: A Review of Notion’s Lore.” Review paper, measured 22 August 2026.
<https://dbms-memory.khe.money>
- [8] Zep AI. “Graphiti — build temporal context graphs for AI agents.” GitHub, Apache-2.0 licence. <https://github.com/getzep/graphiti>
- [9] K. Kulkarni and J.-E. Michels. “Temporal Features in SQL:2011.” *ACM SIGMOD Record*, 41(3):34–43, 2012.
- [10] J. N. Gray, R. A. Lorie, G. R. Putzolu and I. L. Traiger. “Granularity of Locks and Degrees of Consistency in a Shared Data Base.” In *Modelling in Data Base Management Systems*, North Holland, 1976, pp. 365–394.
- [11] O. Benjelloun, H. Garcia-Molina, D. Menestrina, Q. Su, S. E. Whang and J. Widom. “Swoosh: A Generic Approach to Entity Resolution.” *The VLDB Journal*, 18(1):255–276, 2009.
- [12] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan and Y. Zhang. “A-MEM: Agentic Memory for LLM Agents.” arXiv:2502.12110, 2025. <https://arxiv.org/abs/2502.12110>

- [13] Y. Du, W. Huang, D. Zheng, Z. Wang, S. Montella, M. Lapata, K.-F. Wong and J. Z. Pan. “Rethinking Memory in AI: Taxonomy, Operations, Topics, and Future Directions.” arXiv:2505.00675, 2025. <https://arxiv.org/abs/2505.00675>

Harshit Khemani (241302081) · Kush Ahuja · Madhav Bassi · Kushagra Agrawal

B.Tech CSE (AI/ML), Section C · Department of Computer Science & Engineering, SGT University

Phase 2 · submitted 10 September 2026